

مستندات فایل polyp_dataset_persistent.py

PersistentDataset برای پروژه سگمنتیشن پولیپ

پایان نامه کارشناسی ارشد - مهندسی کامپیوتر، گرایش هوش مصنوعی

۷ مهر ۱۴۰۵

فهرست مطالب

۳	۱	مقدمه
۳	۲	بخش ۱: Docstring فایل
۳	۱.۲	ساختار JSON مورد انتظار
۴	۲.۲	مصرف دیسک
۴	۳	بخش ۲: Importها
۵	۴	بخش ۳: کلاس PolypDataset
۵	۱.۴	امضای کلاس و ارث‌بری
۵	۲.۴	متد __init__
۷	۳.۴	متد generate_data_list
۷	۱.۳.۴	مرحله ۱: خواندن JSON
۷	۲.۳.۴	مرحله ۲: انتخاب بخش مناسب
۸	۳.۳.۴	مرحله ۳: فیلتر test (اختیاری)
۸	۴.۳.۴	مرحله ۴: خطای واضح
۸	۴.۴	متد __repr__
۹	۵	بخش ۴: تابع get_polyp_dataset
۹	۱.۵	هدف
۹	۲.۵	نکات کلیدی
۹	۶	بخش ۵: تابع get_test_polyp_dataset
۱۰	۱.۶	هدف
۱۰	۲.۶	نکته‌ی کلیدی: زیرپوشه‌ی cache بر اساس dataset_origin
۱۱	۷	بخش ۶: جریان کامل داده - نمودار
۱۱	۸	بخش ۷: تحلیل عملکرد و مصرف منابع
۱۱	۱.۸	مصرف RAM و دیسک
۱۱	۲.۸	سرعت access repeat vs access first
۱۱	۳.۸	Persistence بین sessionها
۱۲	۹	بخش ۸: نمونه‌های استفاده
۱۲	۱.۹	سناریو ۱: ساخت val و train
۱۲	۲.۹	سناریو ۲: test کل
۱۲	۳.۹	سناریو ۳: فیلترشده test
۱۲	۴.۹	سناریو ۴: حلقه روی ۵ دیتاست (استاندارد مقالات polyp)

۱۳	بخش ۹: محدودیت‌ها و ملاحظات
۱۳	۱.۱۰ محدودیت ۱: سرعت اولیه
۱۳	۲.۱۰ محدودیت ۲: دیسک
۱۳	۳.۱۰ محدودیت ۳: عدم پایداری در Colab/Kaggle
۱۳	۴.۱۰ محدودیت ۴: مدیریت دستی cache
۱۳	بخش ۱۱: جمع‌بندی
۱۳	۱.۱۱ خلاصه
۱۴	۲.۱۱ جدول تصمیم‌گیری نهایی
۱۴	۳.۱۱ نکات طلایی

۱ مقدمه

فایل `polyp_dataset_persistent.py` مسئول تعریف کلاس **Dataset** و توابع سازنده برای پروژه سگمنتیشن پولیپ است. این فایل نسخه‌ی `PersistentDataset` است که برخلاف `CacheDataset`، داده‌های پردازش‌شده را روی دیسک ذخیره می‌کند، نه در RAM.

چرا `PersistentDataset`؟

- `CacheDataset`: داده‌ها در RAM → سریع ولی حافظه‌بر (نیاز به ~۲۲ GB برای `polyp`).
- `PersistentDataset`: داده‌ها روی دیسک → کندتر ولی تقریباً صفر RAM.
- مناسب برای `Kaggle` و `Colab Free` با منابع محدود.

۲ بخش ۱: Docstring فایل

```
1 """
2 Polyp Dataset
3 =====
4
5 This module defines the PolypDataset class for polyp segmentation.
6
7 PolypDataset is built on top of MONAI's PersistentDataset, which caches
8 transformed samples on disk. This is more memory-efficient than
9 CacheDataset but slightly slower due to disk I/O.
10
11 Supported sections:
12     - 'train'      : uses data['fold{N}']['train']
13     - 'validation' : uses data['fold{N}']['val']
14     - 'test'       : uses data['test'] (at root level)
15
16 The 'test' section supports optional filtering by 'dataset_origin'
17 (e.g., 'Kvasir-SEG', 'CVC-ClinicDB', 'CVC-300', ...) for per-dataset
18 evaluation.
19 """
```

توضیح: این docstring در بالای فایل، اطلاعات مهمی را ارائه می‌دهد:

- هدف ماژول: ساخت Dataset برای `polyp`.
- مبنای ارث‌بری: `PersistentDataset` از MONAI.
- تفاوت با **CacheDataset**: cache روی دیسک، نه RAM.
- پشتیبانی از سه section: `train`، `validation`، `test`.
- قابلیت فیلتر: `test` بر اساس `dataset_origin`.

۱.۲ ساختار JSON مورد انتظار

```
1 {
2   "fold0": {
3     "train": [{"image": "...", "mask": "...", "dataset_origin": "..."}, ...],
4     "val":   [{"image": "...", "mask": "...", "dataset_origin": "..."}, ...]
5   },
6   "test": [
7     {"image": "...", "mask": "...", "dataset_origin": "...",
8      "distribution": "..."},
9     ...
10  ]
11 }
```

تحلیل ساختار:

جدول ۱: ساختار JSON برای polyp

بخش	مکان	تعداد (polyp)	کلیدها
train	fold0.train	۱۳۰۵	dataset_origin ,mask ,image
validation	fold0.val	۱۴۵	dataset_origin ,mask ,image
test	root.test	۷۹۸	distribution ,dataset_origin ,mask ,image

۲.۲ مصرف دیسک

```

۱ Disk usage:
۲ The full polyp dataset (train + val + test) consumes approximately
۳ 37.7 GB of disk space when cached (float32). Consider using a
۴ temporary directory (e.g., /kaggle/tmp/polyp_cache) if disk space
۵ is limited.

```

محاسبه‌ی دقیق مصرف دیسک:

بخش	تعداد	حجم تخمینی (float32)
Train	۱۳۰۵	GB ۴.۲۱~
Val	۱۴۵	GB ۳۸.۲~
Test	۷۹۸	GB ۱.۱۳~
جمع	۲۲۴۸	GB ۹.۳۶~

نکته برای انتخاب محل cache:

- Free Colab: /content/polyp_cache (۱۰۸ GB موقت).
- Kaggle: /kaggle/tmp/polyp_cache (۶۰ GB موقت).
- Local: ./data/cache. (بسته به فضای دیسک).

۳ بخش ۲: Import ها

```

۱ import json
۲ from typing import Optional
۳ from dataset.polyp_transforms import get_polyp_transforms
۴ from monai.data.utils import pickle_hashing
۵ from monai.data import PersistentDataset

```

نکته درباره‌ی **pickle_hashing**: MONAI از این تابع برای محاسبه‌ی hash منحصر به فرد هر نمونه استفاده می‌کند. این hash به عنوان نام فایل cache روی دیسک ذخیره می‌شود.

جدول ۲: کاربرد importها

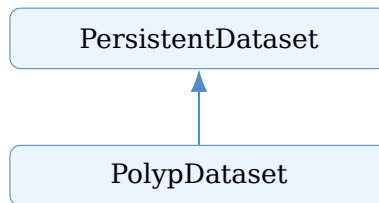
کاربرد	import
خواندن فایل JSON تعریف‌کننده ی split.	json
type hint برای پارامترهای اختیاری.	Optional
ساخت pipelineهای transform از فایل دیگر.	get_polyp_transforms
تابع هش پیش‌فرض MONAI برای PersistentDataset.	pickle_hashing
کلاس پایه از MONAI برای cache روی دیسک.	PersistentDataset

۴ بخش ۳: کلاس PolypDataset

۱.۴ امضای کلاس و ارث‌بری

```
class PolypDataset(PersistentDataset):
```

تحلیل ارث‌بری:



PersistentDataset از MONAI چه می‌کند؟

- اولین access: transform را اجرا و نتیجه را روی دیسک ذخیره می‌کند.
- های access بعدی: مستقیم از دیسک می‌خواند (سریع‌تر).
- Persistence: cache بین sessionها باقی می‌ماند (اگر دیسک پایدار باشد).
- مصرف RAM: تقریباً صفر (فقط لیست مسیرها + یک نمونه).

۲.۴ متد __init__

```

1 def __init__(
2     self,
3     split_path: str,
4     section: str,
5     num_fold: int,
6     transforms,
7     cache_dir: str,
8     dataset_origin: Optional[str] = None,
9     hash_func=None,
10 ):
11     self.section = section
12     self.num_fold = num_fold
13     self.dataset_origin = dataset_origin
14
15     data = self._generate_data_list(split_path)
16     self.num_samples = len(data)
17
18     if hash_func is None:
19         hash_func = pickle_hashing
20 
```

```

۲۱ super().__init__(
۲۲     data=data,
۲۳     transform=transforms,
۲۴     cache_dir=cache_dir,
۲۵     hash_func=hash_func,
۲۶ )

```

توضیح پارامترها:

پارامتر	توضیح
split_path	مسیر فایل JSON تعریف‌کننده ی. split
section	یکی از 'train'، 'validation'، 'test'.
num_fold	شماره fold (فقط برای train/validation).
transforms	Compose MONAI pipeline برای اعمال روی هر نمونه.
cache_dir	پوشه‌ای برای ذخیره‌ی cache روی دیسک.
dataset_origin	(اختیاری) برای فیلتر test.
hash_func	(اختیاری) تابع هش. اگر None باشد، pickle_hashing.

تحلیل مرحله‌به‌مرحله:

۱. ذخیره‌ی attributeها:

```

۱ self.section = section
۲ self.num_fold = num_fold
۳ self.dataset_origin = dataset_origin

```

برای استفاده در generate_data_list و __repr__.

۲. خواندن لیست نمونه‌ها:

```

۱ data = self._generate_data_list(split_path)

```

متد generate_data_list صدا زده می‌شود.

۳. ذخیره‌ی تعداد نمونه:

```

۱ self.num_samples = len(data)

```

۴. تنظیم hash_func پیش‌فرض:

```

۱ if hash_func is None:
۲     hash_func = pickle_hashing

```

نکته‌ی مهم: اگر hash_func=None به PersistentDataset پاس داده شود، در MONAI ۱.۶.۱ خطای TypeError رخ می‌دهد. این if این مشکل را حل می‌کند.

۵. فراخوانی سازنده‌ی پدر:

```

۱ super().__init__(
۲     data=data,
۳     transform=transforms,      # افزودن (transforms)
۴     cache_dir=cache_dir,
۵     hash_func=hash_func,
۶ )

```

تفاوت PersistentDataset API از پارامتر transform (مفرد) استفاده می‌کند، نه transforms (جمع) که در CacheDataset رایج بود.

۳.۴ متد `_generate_data_list`

```

۱ def _generate_data_list(self, split_path: str):
۲     with open(split_path) as fp:
۳         data = json.load(fp)
۴
۵     if self.section == 'train':
۶         datalist = data[f'fold{self.num_fold}']['train']
۷
۸     elif self.section == 'validation':
۹         datalist = data[f'fold{self.num_fold}']['val']
۱۰
۱۱     elif self.section == 'test':
۱۲         datalist = data['test']
۱۳         if self.dataset_origin is not None:
۱۴             datalist = [
۱۵                 sample for sample in datalist
۱۶                 if sample.get('dataset_origin') == self.dataset_origin
۱۷             ]
۱۸         if not datalist:
۱۹             available = sorted({
۲۰                 s.get('dataset_origin') for s in data['test']
۲۱             })
۲۲             raise ValueError(
۲۳                 f"No samples found for dataset_origin="
۲۴                 f"'{self.dataset_origin}'. "
۲۵                 f"Available origins: {available}"
۲۶             )
۲۷     else:
۲۸         raise ValueError(
۲۹             f"Unsupported section: '{self.section}'. "
۳۰             f"Available options are ['train', 'validation', 'test']."
۳۱         )
۳۲     return datalist

```

تحلیل مرحله به مرحله:

۱.۳.۴ مرحله ۱: خواندن JSON

```

۱ with open(split_path) as fp:
۲     data = json.load(fp)

```

کل فایل JSON در RAM بارگذاری می‌شود (حجم ~۶۰۰ KB).

۲.۳.۴ مرحله ۲: انتخاب بخش مناسب

جدول ۳: کلیدهای JSON برای هر section

section	کلید JSON	تعداد polyp
'train'	data['fold0']['train']	۱۳۰۵
'validation'	data['fold0']['val']	۱۴۵
'test'	data['test']	۷۹۸

۳.۳.۴ مرحله‌ی ۳: فیلتر test (اختیاری)

```

۱ if self.dataset_origin is not None:
۲     datalist = [
۳         sample for sample in datalist
۴         if sample.get('dataset_origin') == self.dataset_origin
۵     ]

```

جدول ۴: مقادیر ممکن dataset_origin برای polyp

dataset_origin	تعداد	دسته
Kvasir-SEG	۱۰۰	in_distribution
CVC-ClinicDB	۶۲	in_distribution
CVC-300	۶۰	out_of_distribution
CVC-ColonDB	۳۸۰	out_of_distribution
ETIS-LaribPolypDB	۱۹۶	out_of_distribution

۴.۳.۴ مرحله‌ی ۴: خطای واضح

```

۱ if not datalist:
۲     available = sorted({
۳         s.get('dataset_origin') for s in data['test']
۴     })
۵     raise ValueError(
۶         f"No samples found for dataset_origin='{self.dataset_origin}'. "
۷         f"Available origins: {available}"
۸     )

```

مزیت: اگر کاربر dataset_origin را اشتباه بنویسد، خطای واضح با لیست گزینه‌های موجود می‌گیرد.

۴.۴ متد __repr__

```

۱ def __repr__(self) -> str:
۲     parts = [
۳         f"section='{self.section}'",
۴         f"num_fold={self.num_fold}",
۵     ]
۶     if self.dataset_origin is not None:
۷         parts.append(f"dataset_origin='{self.dataset_origin}'")
۸     parts.append(f"num_samples={getattr(self, 'num_samples', '?')}")
۹     parts.append(f"cache_dir='{self.cache_dir}'")
۱۰    return f"PolypDataset({', '.join(parts)})"

```

هدف: نمایش خوانا از وضعیت dataset در Jupyter/Kaggle
مثال خروجی:

```

۱ PolypDataset(section='train', num_fold=0, num_samples=1305,
۲             cache_dir='/content/polyp_cache/train')
۳
۴ PolypDataset(section='test', num_fold=0, dataset_origin='Kvasir-SEG',
۵             num_samples=100, cache_dir='/content/polyp_cache/test_Kvasir-SEG')

```

۵ بخش ۴: تابع `get_polyp_dataset`

```
1 def get_polyp_dataset(args, sam_image_dim=None):
2     split_path = args['split_path']
3     num_fold = args['n_fold']
4     cache_dir = args['cache_dir']
5
6     train_transforms, val_transforms = get_polyp_transforms(
7         args, sam_image_dim
8     )
9
10    ds_train = PolypDataset(
11        split_path=split_path,
12        section='train',
13        num_fold=num_fold,
14        transforms=train_transforms,
15        cache_dir=f"{cache_dir}/train",
16    )
17    ds_val = PolypDataset(
18        split_path=split_path,
19        section='validation',
20        num_fold=num_fold,
21        transforms=val_transforms,
22        cache_dir=f"{cache_dir}/val",
23    )
24    return ds_train, ds_val
```

۱.۵ هدف

ساخت دو دیتاست (آموزش و اعتبارسنجی) با:

- **train_transforms** برای آموزش (شامل augmentation).
- **val_transforms** برای اعتبارسنجی (بدون augmentation).
- زیرپوشه‌های **cache** جداگانه برای هر بخش.

۲.۵ نکات کلیدی

۱. زیرپوشه‌ی **cache** جداگانه:

```
1 cache_dir=f"{cache_dir}/train"    # زیرپوشه train
2 cache_dir=f"{cache_dir}/val"      # زیرپوشه val
```

این از تداخل بین **cache** **train** و **val** جلوگیری می‌کند.

۲. استفاده از دو **transforms** جداگانه:

- **train_transforms**: با `RandRotate90D`، `RandFlipD`.
- **val_transforms**: بدون augmentation.

۶ بخش ۵: تابع `get_test_polyp_dataset`

```
1 def get_test_polyp_dataset(args, sam_image_dim=None, dataset_origin=None):
2     split_path = args['split_path']
3     num_fold = args['n_fold']
4     cache_dir = args['cache_dir']
5
6     _, val_test_transforms = get_polyp_transforms(args, sam_image_dim)
7
```

```

۸ # Build a separate cache subdir per dataset_origin (if filtering).
۹ if dataset_origin is not None:
۱۰     safe_name = dataset_origin.replace('/', '_').replace(' ', '_')
۱۱     subdir = f"test_{safe_name}"
۱۲ else:
۱۳     subdir = "test_all"
۱۴
۱۵ ds_test = PolypDataset(
۱۶     split_path=split_path,
۱۷     section='test',
۱۸     num_fold=num_fold,
۱۹     transforms=val_test_transforms,
۲۰     cache_dir=f"{cache_dir}/{subdir}",
۲۱     dataset_origin=dataset_origin,
۲۲ )
۲۳ return ds_test

```

۱.۶ هدف

ساخت یک دیتاست تست با قابلیت:

- کل test (۷۹۸ نمونه) اگر dataset_origin=None.
- یک دیتاست خاص اگر dataset_origin داده شود.

۲.۶ نکته‌ی کلیدی: زیرپوشه‌ی cache بر اساس dataset_origin

```

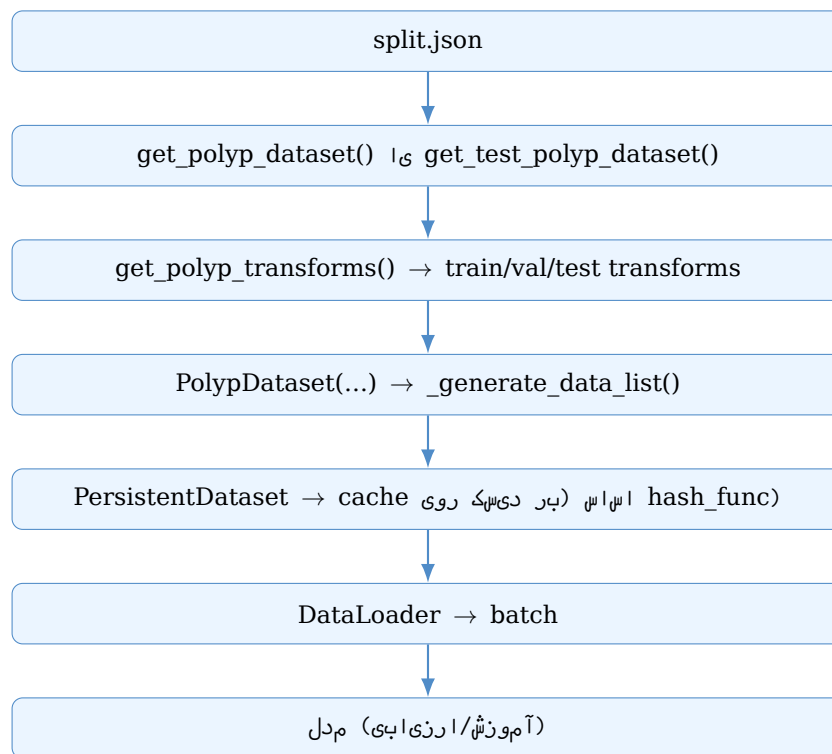
۱ if dataset_origin is not None:
۲     safe_name = dataset_origin.replace('/', '_').replace(' ', '_')
۳     subdir = f"test_{safe_name}"
۴ else:
۵     subdir = "test_all"

```

چرا زیرپوشه‌ی جداگانه؟ اگر هم test کل و هم test فیلترشده از یک cache استفاده کنند، ممکن است cache با هم مخلوط شود. زیرپوشه‌ی جداگانه این مشکل را حل می‌کند.

مثال از زیرپوشه‌ها:

dataset_origin	زیرپوشه‌ی cache
None	/content/polyp_cache/test_all
'Kvasir-SEG'	/content/polyp_cache/test_Kvasir-SEG
'CVC-ClinicDB'	/content/polyp_cache/test_CVC-ClinicDB
'CVC-300'	/content/polyp_cache/test_CVC-300
'CVC-ColonDB'	/content/polyp_cache/test_CVC-ColonDB
'ETIS-LaribPolypDB'	/content/polyp_cache/test_ETIS-LaribPolypDB



شکل ۱: جریان کامل داده در persistent dataset

جدول ۵: مقایسه‌ی PersistentDataset و CacheDataset

معیار	CacheDataset	PersistentDataset
محل cache	RAM	دیسک
مصرف RAM (Train)	~ ۲۲ GB	~ ۲۱ MB
مصرف دیسک	۰	~ ۹.۳۶ GB
سرعت access first	سریع	کند
سرعت های access بعدی	سریع‌ترین	سریع
Persistence	×	✓

۷ بخش ۶: جریان کامل داده – نمودار

۸ بخش ۷: تحلیل عملکرد و مصرف منابع

۱.۸ مصرف RAM و دیسک

۲.۸ سرعت access repeat vs access first

مهم: پس از warmup اولیه (اولین epoch آموزش با سرعت نزدیک به CacheDataset انجام می‌شود، ولی بدون مصرف RAM بالا).

۳.۸ Persistence بین session ها

• اگر cache در /content/polyp_cache باشد → با disconnect پاک می‌شود.

جدول ۶: سرعت PersistentDataset در حالت‌های مختلف

عملکرد	زمان	توضیح
(build) access First	~ ۱۵۹ ms	خواندن + transform + ذخیره
(read) access Repeat	~ ۸.۵ ms	خواندن مستقیم از دیسک
Speedup	~ ۲۷×	سریع‌تر در های access بعدی

- اگر cache در `/content/drive/MyDrive/polyp_cache` باشد → پایدار (ولی کندتر).
- اگر cache در `/kaggle/working` باشد → پایدار (ولی محدود به ۲۰ GB).
- اگر cache در `/kaggle/tmp` باشد → پاک می‌شود (ولی ۶۰ GB دارد).

۹ بخش ۸: نمونه‌های استفاده

۱.۹ سناریو ۱: ساخت train و val

```

1 args = {
2     'split_path': '/content/polyp_sam_less/data/split.json',
3     'n_fold': 0,
4     'cache_dir': '/content/polyp_cache',
5     'image_key': 'image',
6     'mask_key': 'mask',
7 }
8
9 ds_train, ds_val = get_polyp_dataset(args, sam_image_dim=1024)
10 print(ds_train)
11 # PolypDataset(section='train', num_fold=0, num_samples=1305,
12 #               cache_dir='/content/polyp_cache/train')
13 print(ds_val)
14 # PolypDataset(section='validation', num_fold=0, num_samples=145,
15 #               cache_dir='/content/polyp_cache/val')

```

۲.۹ سناریو ۲: test کل

```

1 ds_test = get_test_polyp_dataset(args, sam_image_dim=1024)
2 print(ds_test)
3 # PolypDataset(section='test', num_fold=0, num_samples=798,
4 #               cache_dir='/content/polyp_cache/test_all')

```

۳.۹ سناریو ۳: test فیلترشده

```

1 ds_kvasir = get_test_polyp_dataset(
2     args, sam_image_dim=1024, dataset_origin='Kvasir-SEG'
3 )
4 print(ds_kvasir)
5 # PolypDataset(section='test', num_fold=0, dataset_origin='Kvasir-SEG',
6 #               num_samples=100,
7 #               cache_dir='/content/polyp_cache/test_Kvasir-SEG')

```

۴.۹ سناریو ۴: حلقه روی ۵ دیتاست (استاندارد مقالات polyp)

```

1 DATASETS = ['Kvasir-SEG', 'CVC-ClinicDB', 'CVC-300',
2             'CVC-ColonDB', 'ETIS-LaribPolypDB']
3
4 for name in DATASETS:
5     ds = get_test_polyp_dataset(args, img_dim, dataset_origin=name)
6     loader = DataLoader(ds, batch_size=1, shuffle=False, num_workers=0)
7     metrics = evaluate(model, loader)
8     print(f"{name:25s}: Dice={metrics['dice']:.4f}")

```

خروجی مورد انتظار:

```

1 Kvasir-SEG           : Dice=0.9182
2 CVC-ClinicDB         : Dice=0.9251
3 CVC-300              : Dice=0.8743
4 CVC-ColonDB          : Dice=0.7824
5 ETIS-LaribPolypDB    : Dice=0.7862

```

۱۰ بخش ۹: محدودیت‌ها و ملاحظات

۱.۱۰ محدودیت ۱: سرعت اولیه

اولین epoch کندتر است چون cache در حال ساخت است. برای train با ۱۳۰۵ نمونه، این ممکن است ۱۰-۱۵ دقیقه طول بکشد.

۲.۱۰ محدودیت ۲: دیسک

- Colab Free: ۱۰۸ GB → کافی.
- Kaggle: ۶۰ GB در tmp → کافی.
- Local: بستگی به فضای دیسک دارد.

۳.۱۰ محدودیت ۳: عدم پایداری در Colab/Kaggle

- Colab: با disconnect، cache پاک می‌شود.
- Kaggle: با پایان tmp session، پاک می‌شود.
- راه‌حل: برای cache persistence، Google Drive یا /kaggle/working بگذار.

۴.۱۰ محدودیت ۴: مدیریت دستی cache

نکته: برای جلوگیری از پر شدن دیسک، cache را به صورت دوره‌ای پاک کن:

```

import shutil
shutil.rmtree('/content/polyp_cache')

```

۱۱ بخش ۱۰: جمع‌بندی

۱.۱۱ خلاصه

- کلاس **PolypDataset**: نسخه‌ی PersistentDataset برای polyp.
- مزیت اصلی: مصرف RAM تقریباً صفر.

- عیب اصلی: مصرف دیسک ~۳۷ GB.
- مناسب برای: Kaggle، Colab Free، سیستم‌های با RAM محدود.

۲.۱۱ جدول تصمیم‌گیری نهایی

سناریو	CacheDataset	PersistentDataset
GB $32 \geq$ RAM	✓	△
GB $32-16$ RAM	△	✓
Free) (Colab GB 12 RAM	×	✓
Kaggle	△	✓
دیسک کم	✓	×

۳.۱۱ نکات طلایی

۵ نکته‌ی کلیدی:

۱. PersistentDataset برای RAM محدود بهترین گزینه است.
۲. hash_func باید صریحاً تنظیم شود (pickle_hashing).
۳. هر بخش (train/val/test) زیرپوشه‌ی cache جداگانه دارد.
۴. dataset_origin امکان گزارش per-dataset می‌دهد.
۵. اولین epoch کندتر است، بعد از آن سریع می‌شود.